

Computing quasiparticle braids using estimation of distribution algorithms

R. Santana¹, R. B. McDonald², H. G. Katzgraber^{2,3}.

the date of receipt and acceptance should be inserted later

Abstract Topological quantum computing is an alternative framework for avoiding the quantum decoherence problem in quantum computation. The problem of executing a gate in this framework can be posed as the problem of braiding quasiparticles and it can be reduced to finding an optimal product of braid generators where the optimality is defined in terms of the gate approximation and the braid's length. Recent results have shown that it is possible to use evolutionary algorithms to find optimal braids. In this paper we show that estimation of distribution algorithms (EDAs) can be an efficient alternative to deal with this problem. We propose a number of enhancements to EDAs and compare our results with other state of the art approaches to the problem. The behavior of the algorithm is investigated in the approximation of several quantum gates with different characteristics. The introduced algorithm obtains solutions with an accuracy in the order of 10^{-6} , and lengths up to 9 times shorter than those expected from braids of the same accuracy obtained with other methods. Our algorithm obtains more accurate approximation of the bases of the icosahedral group than those obtained by brute-force search for shorter braids.

keywords: evolutionary computation, braid optimization, topological computing, probabilistic graphical models, EDAs

1 Introduction

Quantum computation systems [21] are based on the idea of exploiting the quantum-mechanical properties of the matter to efficiently manipulate infor-

¹Department of Computer Science and Artificial Intelligence. University of the Basque Country (UPV/EHU). P. Manuel de Lardizabal, 20018, Guipuzcoa. Spain

²Department of Physics and Astronomy, Texas A&M University, College Station, Texas 77843-4242, USA.

³Santa Fe Institute, 1399 Hyde Park Road, Santa Fe, NM 87501, USA
roberto.santana@ehu.es, katzgraber@physics.tamu.edu

mation. An obstacle for the construction of large quantum computers is the problem of quantum decoherence, that can be viewed as the loss of information of the quantum system due to the interaction with the environment. One possible solution to this problem is the design of quantum systems immune to quantum decoherence on a hardware level.

Topological quantum computing (TQC) [3,13,31] investigates quantum computing systems that, given the properties of the quasiparticles they use, are not affected by quantum decoherence. The key idea of these systems is that quantum information can be stored in global properties of the system and thus affected only by global operations but not by local perturbations such as noise. In TQC, quantum gates are carried out by adiabatically braiding quasiparticles around each other. This braiding is used to perform the unitary transformations of a quantum computation.

One of the essential questions to design a TQC is to find a product of braid generators (matrices) that approximates a quantum gate with the smallest error and as short as possible to prevent loss [16]. The relevant question of minimizing the error of a TQC design can be posed as a braid optimization problem. Some optimization approaches to this question have been proposed. Exhaustive search [3] has been applied to search for braids of manageable size. Other methods such as the Solovay-Kitaev algorithm [7] provide bounds on the accuracy and length of the braids. However, they do not allow the user to tune the balance between the accuracy and the length as pioneered in [16] where the use of genetic algorithms (GAs) to find optimal braids is proposed.

Estimation of distribution algorithms (EDAs) [15,20,14,33] are evolutionary algorithms (EAs) that apply learning and sampling of distributions instead of classical crossover and mutation operators. Modeling the dependencies between the variables of the problem can serve to efficiently orient the search to more promising areas of the search space by explicitly capturing and exploiting potential relationships between the problem variables. In addition, the information contained in the models can reveal previously unknown information about the problem. EDAs have been applied applied to a variety of real-world optimization problems including problems from Physics [22,24,34].

In this paper, we build on the GA approach introduced in [16] and propose the use of EDAs for solving the braid optimization problem. We use the fitness function proposed in [16] and introduce a new representation, variation operators and enhancement procedures in the framework of EDAs. Furthermore, we conduct an exhaustive investigation of our algorithm on a large number of target quantum gates. For example, we evaluate the ability of the EDAs to approximate the 60 unitary matrices defining the icosahedral group [5] and show that the EDAs can produce solutions with a lower error for these gates. We remark that the braid optimization problem is also relevant to investigate the performance of EAs for functions that are the product of its arguments. In comparison to additive functions this class of “product functions” have been seldom used as function benchmarks in evolutionary computation.

The paper is organized as follows: The next section briefly describes braids and Fibonacci anyons [3]. The problem formulation, including the problem

representation and fitness functions used are presented in Section 3. Section 4 introduces the different variants of EDAs proposed for the target gate approximation problem. Work related to our approach is discussed in Section 5. The experimental framework to evaluate our proposal is described in Section 6 where the numerical results of our experiments are also presented. The main contributions of the paper are summarized in Section 7 where some lines for future research are also discussed.

2 Braids and anyons

2.1 Anyons

Qubits play in quantum computation a role similar to that played by bits in digital computers. Anyons appear as emergent quasiparticles in fractional quantum Hall states and as excitations in microscopic models of frustrated quantum magnets that harbor topological quantum liquids [26]. In a topological computer, the qubits are represented by sets of anyons. By lining up pairs of anyons and swapping adjacent anyons following a specific sequence it is possible to manipulate them. The set of sequential swaps can be represented using a “braid”. Fibonacci anyons are the simplest anyons with non-Abelian braiding statistics that can give rise to universal quantum computation. Fibonacci anyon braids [3] only encompasses one-qubit gates.

2.2 Braids

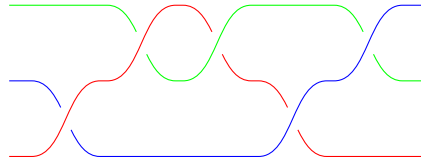


Fig. 1 A braid with three strands.

A braid [12] is an embedding of a collection of strands that have their ends in two rows of points that are set one above the other with respect to a choice of vertical. The strands are not individually knotted and they are disjoint from one another. Figure 1 shows an example of a braid with three strands.

A braid operation can be represented by a matrix that acts on the qubit space. These matrices are referred to as generators, and the quantum gate that a braid represents is the product of the generators that encode the individual braid operations. The braid group B_n is defined by a set of generators

$\sigma_1, \dots, \sigma_n$ that satisfy the following relationships:

$$\sigma_i \sigma_k = \sigma_k \sigma_i \text{ if } |i - k| \geq 2 \quad (1)$$

$$\sigma_i \sigma_{i+1} \sigma_i = \sigma_{i+1} \sigma_i \sigma_{i+1} \quad (2)$$

We work with braids of three strands that can be represented by two generators. Let σ_1 and σ_2 represent two possible generators. σ_1^{-1} and σ_2^{-1} respectively represent their inverses.

$$\sigma_1 = \begin{pmatrix} e^{-\frac{i7\pi}{10}} & 0 \\ 0 & -e^{-\frac{i3\pi}{10}} \end{pmatrix} \quad \sigma_2 = \begin{pmatrix} -\tau e^{-\frac{i\pi}{10}} & -i\sqrt{\tau} \\ -i\sqrt{\tau} & -\tau e^{-\frac{i\pi}{10}} \end{pmatrix} \quad (3)$$

where $\tau = \frac{\sqrt{5}-1}{2}$.

Given a braid B , $len()$ is a function that returns the braid's length l (e.g. $B = \sigma_1 \sigma_1 \sigma_2 \sigma_1^{-1}$, $l = len(B) = 4$). Since the product of a matrix by its inverse reduces to the identity matrix, some braids can be simplified reducing their length. Therefore, we also define function $elen()$, that has a braid as its argument and returns the braid's *effective length* which is the length of braid after all possible simplifications have been conducted.

$$\sigma_1 \sigma_1 \sigma_1 \sigma_1^{-1} = \sigma_1 \sigma_1 \sigma_1 \quad (4)$$

$$\sigma_2^{-1} \sigma_1 \sigma_1 \sigma_1^{-1} \sigma_1^{-1} \sigma_2 \sigma_1^{-1} = \sigma_1^{-1} \quad (5)$$

In the braids shown in examples (4) and (5), the effective length values are 3 and 1, respectively.

Let T represent the target matrix (gate to be emulated), the braid error is calculated with the following metric [16]:

$$\epsilon = |B - T| \quad (6)$$

where the matrix norm used is

$$|M| = \sqrt{\sum_{ij} M_{ij}^2}. \quad (7)$$

The problem of finding braiding operations that approximate gates is then reduced to finding a product chain of the reduced generators and their inverses that approximates the matrix representing the quantum gate. Two elements that describe the quality of a braid are its error ϵ and its length l .

In this paper we address the problem of finding a product of generator matrices that approximates a given target T . The search for the product of generators is approached as a combinatorial problem defined in the space of all possible braids of size n . Although the methodology we propose can be extended to other braids, we focus on anyon braids since they are one of the best known in TQC [16, 37].

3 Problem formulation

Let $\mathbf{X} = (X_1, \dots, X_n)$ denote a vector of discrete random variables. We use $\mathbf{x} = (x_1, \dots, x_n)$ to denote an assignment to the variables. I denotes a set of indices in $\{1, \dots, n\}$, and X_I (respectively x_I) a subset of the variables of $\mathbf{X}$ (respectively $\mathbf{x}$) determined by the indices in I .

In our representation for the quasiparticle braids problem, $\mathbf{X} = (X_1, \dots, X_n)$ represents a braid of length n , where X_i takes values in $\{0, 1, \dots, 2g-1\}$ and g is the number of generators. Given an order for the generators $\sigma_1, \sigma_2, \dots, \sigma_g$, $X_i = j, j < g$ means that the matrix in position i is σ_{j+1} . If $X_i = j, j \geq g$, then the matrix in position i is $\sigma_{(j-g)+1}^{-1}$. For example, for generators shown in Equation (3), and $B = \sigma_1 \sigma_1 \sigma_2 \sigma_2^{-1} \sigma_1^{-1}$, the corresponding braid representation is $\mathbf{x} = (0, 0, 1, 3, 2)$. Notice that this is a fixed length representation.

We are interested in the solution of an optimization problem formulated as $\mathbf{x}^* = \operatorname{argmax}_{\mathbf{x}} f(\mathbf{x})$, where $f : S \rightarrow R$ is called the objective or fitness function. The optimum $\mathbf{x}^*$ is not necessarily unique. To evaluate the fitness function associated to a solution $\mathbf{x}$, firstly the product of braid matrices B is computed according to $\mathbf{x}$ and then the error ϵ is calculated from B as in Equation (6).

The fitness function [16] is defined as:

$$f(\mathbf{x}) = \frac{1 - \lambda}{1 + \epsilon} + \frac{\lambda}{l} \quad (8)$$

where l is the braid's length, and λ serves to balance the two conflicting goals, i.e., having short braids or low approximation error. When $\lambda = 0$, braids are optimized only for the error and the function reaches its maximum value when this error is minimized.

We define functions $\hat{f}(\mathbf{x})$ and $\bar{f}(\mathbf{x})$ as two variations of function (8). Function $\hat{f}(\mathbf{x})$ is identical to $f(\mathbf{x})$, except that the effective length $\hat{l} = \operatorname{elen}(B)$ is used instead of the braid's length. Function $\bar{f}(\mathbf{x})$ outputs the maximum value of the function for any of the braids contained in B that start from the first position, i.e.

$$\bar{f}(\mathbf{x}) = \max_{\mathbf{y}, \mathbf{y} \in \{(x_1), (x_1, x_2), \dots, (x_1, \dots, x_i), \dots, (x_1, \dots, x_n)\}} f(\mathbf{y}) \quad (9)$$

4 Estimation of distribution algorithms

EDAs assume that some statistical regularities in the best solutions exist and that these regularities can be used to bias the search to the promising areas of the search space. Therefore, it is important to determine whether such assumption holds for the braid problem. In [30], we have investigated whether possible regularities of the search space for the braid problem can be detected from the statistical analysis of the Boltzmann distribution [18] defined on the braid function. Similar analysis has been successfully applied to investigate

the dependencies that arise in the configurations of simplified protein models [29] and conductance-based neuron models [27].

Our analysis showed that there are at least two types of regularities of the braid problem that are translated into statistical features. Firstly, there are different frequencies associated to the generators in the space of the best solutions. Secondly, there are strong dependencies between the variables, particularly those that are adjacent in the braid representation. This evidence can be useful to explain the results of different variants of EDAs for this problem. In this section we describe the main characteristics of the EDAs used to solve the braid problem.

4.1 EDA approach

The pseudocode of a general EDA is shown in Algorithm 1. The organization of the algorithm and the evaluation and selection steps are similar to a genetic algorithm. The main differences are in step 5 the algorithm, where a probabilistic model is learned from the data, and in step 6, where the model is used to sample the new solutions of the population. We use as a termination criterion a maximum number of generations. Truncation selection was used, in which the best 5% of the population is selected according to the fitness values. Finally, in order to form the new population, the new sampled solutions are combined with the set of best solutions (elitist solutions) selected from the current generation and the worst solutions are removed.

Algorithm 1: **Estimation of distribution algorithm**

```

1  Set  $t \leftarrow 0$ . Generate  $N$  solutions randomly.
2  do {
3    Evaluate the solutions using the fitness function.
4    Select a population  $D_t^S$  of  $K \leq N$  solutions according to a selection method.
5    Calculate a probabilistic model of  $D_t^S$ .
6    Generate  $N$  new solutions sampling from the distribution represented in the model
    and combine with current population.
7     $t \leftarrow t + 1$ 
8  } until Termination criteria are met.
```

4.2 Probabilistic models

One relevant question for solving combinatorial problems using EDAs is which is the type and amount of interactions between the variables of the problem that should be modeled in order to find the optimal solutions. Different probabilistic models are able to represent different number of interactions but usually, as this capacity increases, the computational cost of the algorithm

also grows. We investigate three EDAs that differ in the probabilistic models they use and that have been previously applied to a variety of combinatorial problems and [1, 2, 8, 9, 29]. It follows a description of the models:

We work with positive distributions denoted by p . $p(x_I)$ denotes the marginal probability for $\mathbf{X}_I = \mathbf{x}_I$. $p(x_i | x_j)$ denotes the conditional probability distribution of $X_i = x_i$ given $X_j = x_j$. Three types of probabilistic graphical models are defined: 1) Univariate model. 2) 1-order Markov model. 3) Tree model.

In the univariate model variables are considered to be independent, and the probability of a solution is the product of the univariate probabilities for all variables:

$$p_u(\mathbf{x}) = \prod_{i=1}^n p(x_i) \quad (10)$$

We call the EDA that uses the univariate probabilistic model as univariate marginal distribution algorithm (UMDA) [20].

In the 1-order Markov model, the configuration of variable X_i depends on the configuration of the previous variable in the solution:

$$p_{MK}(\mathbf{x}) = p(x_1) \prod_{i=2}^n p(x_i | x_{i-1}) \quad (11)$$

The EDA that uses the 1-order Markov model is called in this paper Mk-EDA.

A probability distribution $p_{\mathcal{T}}(\mathbf{x})$ that is conformal with a tree is defined as:

$$p_{\mathcal{T}}(\mathbf{x}) = \prod_{i=1}^n p(x_i | pa(x_i)), \quad (12)$$

where $Pa(X_i)$ is the parent of X_i in the tree, and $p(x_i | pa(x_i)) = p(x_i)$ when $pa(X_i) = \emptyset$, i.e. X_i is a root of the tree. We allow the existence of more than one root in the PGM (i.e. forests) although for convenience of notation we refer to the model as tree.

The EDA that uses the tree model is called in this paper Tree-EDA.

Univariate approximations are expected to work well for functions that can be additively decomposed into functions of order one (e.g. $g(\mathbf{x}) = \sum_i x_i$). However, other non additively decomposable functions can be easily solved with EDAs that use univariate models (e.g. $g(\mathbf{x}) = \prod_i x_i + \sum_i x_i$) [19]. Therefore, it makes sense to test the univariate approximation for the braid problem.

The 1-order Markov model captures only dependencies between adjacent variables. Learning of the model implies learning the conditional pairwise dependence relationships between the adjacent variables. During the sampling process, first variable X_1 is sampled, and every other variable X_i is sampled conditioned on X_{i-1} .

The tree model can represent a maximum of $n - 1$ bivariate dependencies and is the only model among the three for which the structure of the dependencies between the variables has to be learned from the data. The construction

of the tree structure from data implies the detection of the most important bivariate interactions between the variables. Initially, the univariate and bivariate probabilities are respectively calculated for every variable and pair of variables. From these marginal probabilities, the mutual information between each pair of variables is computed. To construct the tree structure, the Chow-Liu algorithm [6] is used. It finds the maximum weight spanning tree from the matrix of mutual information between pairs of variables. Probabilistic logic sampling (PLS) [10] is applied to sample new solutions from the tree.

The computational cost of EDAs is mainly associated to the methods needed to learn and sample the models. The most complex EDA used in this paper is Tree-EDA which has a computational cost $O(n^2)$. Examples of EDAs that use univariate, 1-order Markov, and tree models are respectively presented in [20], [8] and [2, 29] and details on the methods used to learn and sample the models can be obtained from these references.

4.3 Enhancements to the EDAs

We consider three enhancements to EDAs: 1) Use of a local optimizer. 2) Partial sampling. 3) Recoding.

As is the case of other EAs, EDAs can be enhanced by the incorporation of local optimizers [23, 28]. We use a greedy optimization algorithm that is applied during the evaluation of the population by the EDA. The algorithm starts from the solution generated by the EDA. In each iteration, the local optimizer evaluates all the $3n$ solutions that are different to the current solution in only one variable (the neighbor solutions). The next selected solution is the neighbor that improves the fitness of the current solution the most. The algorithm stops when none of the neighbors improves the fitness of the current solution.

During the sampling step of an EDA, all variables are assigned their values according to the probabilistic model and the sampling method. For the EDA that uses the univariate model, variables are independently sampled. For 1-order Markov and tree, PLS [10] is used. In both methods, all variables are assigned the new values. However, for some problems with higher-order interactions using a base-template solution can be better than generating each new solution from scratch.

In partial sampling [25, 36], a solution of the population is selected and only a subset of its variables are sampled according to the model. We use two variants of partial sampling I) Partial sampling where the number of variables to be modified is randomly selected between 1 and n . II) Partial sampling, where the number of variables to be modified is randomly selected between 1 and $\frac{n}{2}$.

Recoding consists in modifying the representation of the solution after the fitness evaluation. For functions f and $\bar{f}$ it is possible to recode the solution by eliminating redundant generators (e.g., pairs $\sigma_i \sigma_i^{-1}$). The rationale of using recoding is that meaningful variables will be located closer to the beginning of the braid. Since solutions have a fixed length, the last variables will be kept

unused, i.e. garbage information. Therefore, we devised two ways to fill these gaps: I) Leaving the unused variables as they were in the original solution. II) Replacing the unused variables by a reverse copy of the variables used in the evaluation. The second variant intends to replicate information that has proved to be “informative” about the problem. Equations (13) and (14) show examples of recoding type I and II, respectively. In this hypothetical examples, the underlined variables are those that provided the best fitness after simplifying the braid and evaluating function $\tilde{f}$.

$$(0, \underline{3}, 1, 3, \underline{3}, \underline{3}, 2, 1, 2, 2) = (0, 3, 3, 3, \underline{2}, 3, 2, 1, 2, 2) \quad (13)$$

$$(0, \underline{3}, 1, 3, \underline{3}, \underline{3}, 2, 1, 2, 2) = (\underline{0}, 3, 3, 3, \underline{2}, 2, 3, 3, 3, 0) \quad (14)$$

5 Related work

In addition to genetic algorithms [16], other approaches have been applied to the quantum gate approximation problem.

5.1 Solovay-Kitaev algorithm

The Solovay-Kitaev algorithm [13, 7] is a constructive procedure that guarantees to obtain a braid approximation of a single-qubit gate with an arbitrary error ϵ . The algorithm uses as input a library set of gates $\{L_1, L_2, \dots, L_k\}$ and recursively constructs a combination of these gates that minimizes the error until achieving the desired error. The Solovay-Kitaev theorem states [35] that, for a given gate G , error ϵ , and library set $\{L_i\}$, there always exists a sequence of library gates $(\prod_{j=1}^l A_j | A_j \in L_i L_i^{-1})$ such that [7]:

$$\|G - \prod_l A_l\| \leq \epsilon \quad (15)$$

Furthermore, it is possible to set the dependence between the length of the sequence (l) and ϵ as $l = O[\log^c(1/\epsilon)]$, where the Solovay-Kitaev standard algorithm gives a scaling of sequence size according to $c = 3.97$.

One drawback of the standard Solovay-Kitaev algorithm is that in each iteration the braid length increases by a factor of 5. As a result, there may be shorter braids that achieve the same precision of the braid produced by the algorithm. Another factor that influences the behavior of the standard Solovay-Kitaev algorithm is that, at its first layer, it conducts an exhaustive search over the space of braids up to length l_0 . Obviously, such sort of exhaustive search is only feasible for small values of l_0 . Recently, improved variants of the standard Solovay-Kitaev algorithm have been proposed [35].

5.2 Geometrical approaches

Other approaches [5,4,17] try to construct an initial braid approximation of the target gate and then refine this approximation. The way refinement is conducted is similar to the idea of hashing as applied for data indexing in computer science. The key question is how to define the hashing table that allows to efficiently improve the braid approximations.

We present in some more detail the quantum hashing method since one of the problems addressed in the section of experiments is inspired in this work.

5.2.1 Quantum hashing with the icosahedral group

Braid generators are unitary matrices, i.e. square matrices U that satisfy that $U^H = U^{-1}$, where U^H denotes the conjugate transpose and U^{-1} is the matrix inverse. The special unitary group $SU(2)$ is the group of all 2×2 unitary matrices with determinant 1. The 3d rotation group $SO(3)$ is the group of all rotations about the origin of three-dimensional Euclidean space $\mathcal{R}^3$ under the operation of composition. The quantum hashing procedure presented in [5,4] uses the homomorphism between $SU(2)$ and $SO(3)$ to associate a 2×2 unitary matrix to each element of the icosahedral rotation group $\mathcal{I} = \{g_1, \dots, g_{60}\}$.

The icosahedral group comprises the 60 rotations around the axes of symmetry of the icosahedron. Each of these rotations is initially approximated by a braid of fixed length L that is searched using brute-force. The resulting approximation of the group $\mathcal{I}$ is then defined as $\tilde{\mathcal{I}} = \{\tilde{g}_1, \dots, \tilde{g}_{60}\}$. The elements of $\tilde{\mathcal{I}}$ are then used to find a combination of m elements that produces a rough braid representation of any target quantum gate T . Finally, the rough approximation of T is refined by conducting another search in another set of braid approximations found by applying brute-force in the search space of larger braids. In our experiments we use EDAs to find braid approximations of all the elements of $\mathcal{I}$.

5.3 Brute-force methods

Brute-force has been extensively applied to search for braids of manageable size. As mentioned before, the Solovay-Kitaev algorithm and the quantum hashing method apply a preliminary exhaustive search to find the initial qubit approximations. For instance, in [3] brute-force is used to optimize braids of length up to 46. The quantum hashing method presented in [5] applies brute-force to find braids of $L = 24$, and in [4] the 60 elements of the icosahedral group are approximated with braids up to length $L = 68$. The high computational cost of such an exhaustive search is justified because once the braids are found they may be reused many times to approximate other targets. However, it is clear that brute-force is not feasible for higher L .

It is remarkable that, with the exception of GAs, other combinatorial optimization algorithms have not been applied as part of the Solovay-Kitaev

algorithm and the quantum hashing method to create more accurate initial braids of medium and large size. Combinatorial optimization methods allow the user to tune the balance between the error and the length of the braid and provide different solutions to the problem.

5.4 Application of EDAs to problems from Physics

Other problems from physics have been previously treated with EDAs. In particular, spin glass models with different types of interactions and topologies have been addressed [22, 24, 34, 32]. Two important differences between the braid problem and the spin glass models that makes it particularly challenging is that its fitness function is multiplicative and that the representation is non binary. In fact the cardinality of the variables can increase with the number of generators.

6 Experiments

The main objective of our experiments is to evaluate the capacity of the EDAs to find optimal solutions to the quantum gate approximation problem. A second objective is to evaluate different variants of the algorithm and compare their results to other methods.

6.1 Experimental settings

Each EDA is characterized by 5 parameters:

- Use of local optimizer. 0: Only EDA is applied, 1: EDA is combined with greedy search as described in Section 4.3.
- Type of function and representation. 0: Function f , 1: Function $\bar{f}$ without recoding, 2: Function $\bar{f}$ with recoding I, 3: Function $\bar{f}$ with recoding II.
- λ value. 0:0.0, 1:0.01, 2:0.05, 3:0.1.
- Sampling method. 0: Normal, 1: Partial sampling I, 2: Partial sampling II.
- Type of probabilistic model. 0: Univariate, 1: 1-order Markov, 2: Tree.

The experimental settings were selected to investigate different aspects that influence the behavior of the algorithm. For the first problem addressed, we run experiments for $n \in \{50, 100, 150, 200, 250\}$ in order to evaluate the scalability of the algorithms. Increasing n may lead to obtain braids with a smaller error.

The total number of variants of the algorithm was $2 \times 4 \times 4 \times 3 \times 3 = 288$. All the algorithms use truncation selection, in which the best 5% of the population is selected. EDAs that do not incorporate the greedy local search use a population size $N = 10000$. For these EDAs, the number of generations was dependent on n as $N_g = 15n$. Due to the large number of evaluations

spent by the greedy search method, the population size for all hybrid EDAs was $N = 100n$ and the number of generations was fixed to $N_g = 100$. For each EDA variant, 100 experiments were run.

6.2 Problem benchmark

Target gates define different instances of the braid problem. Additionally, the maximum possible length of the braid solutions behave as constraints for the precision of the best approximation. In our experiments we use different target gates and different braid lengths.

The first target (T_1) was the one used in [16] to evaluate the performance of the introduced GA.

$$T_1 = \begin{pmatrix} 0 & i \\ i & 0 \end{pmatrix} \quad (16)$$

The results obtained on target T_1 allowed us to identify the best EDA variant which was then applied to another set of 60 target gates, those unitary matrices that belong to the icosahedral group $\mathcal{I}$ explained in Section 5.2.1. The definition and implementation of the group elements were obtained from the original code¹ available from the authors of [5, 4].

6.3 Results for target T_1

Figure 2a) and Table 1 respectively show the parameters that characterize the best braids found by the EDAs for each value of n , and the braids. In Figure 2a), we also show an estimate of the length of the braids ($O[\log_{10}^{3.97}(1/\epsilon)]$) that would compute the Solovay-Kitaev algorithm [7] to obtain the same error ϵ of our best solutions. The lengths of our solutions clearly outperform these estimates. Figure 2b) shows the length of all the best solutions achieved for each value of n . It can be observed in Figure 2 that EDAs are able to find several braids with different lengths for $n = 150$ and $n = 200$.

6.3.1 Behavior of the EDA variants

We further investigate the behavior of the different EDA variants. Figure 3 shows the violin plots [11] with the distribution of the best values found in all the executions for: a) All EDA variants without local optimizer (14400 runs) and b) EDAs with local optimizer, recoding type II, and that use partial sampling type II (300 runs). Each violin plot shows a histogram smoothened using a kernel density with Normal kernel. The mean and median are shown as red crosses and green squares, respectively.

In Figure 3, the modes of the Normal distribution indicate the existence of a local optimum of the error with a very wide basin of attraction. This local

¹ <https://sites.google.com/site/braidanyons/download>

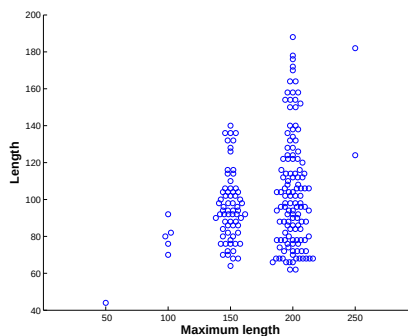


Fig. 2 Target T_1 : a) Parameters of the best braids found by the EDAs for each value of n .
b) Length of the best solutions found for each value of n .

n	braid
50	$\sigma_1^{-1}\sigma_2^3\sigma_1^{-2}\sigma_2\sigma_1^{-1}\sigma_2\sigma_1^{-2}\sigma_2^3\sigma_1^{-1}\sigma_2\sigma_1^{-1}\sigma_2\sigma_1^{-2}\sigma_2\sigma_1^{-2}\sigma_2\sigma_1^{-1}\sigma_2^3\sigma_1^{-1}\sigma_2^2\sigma_1^{-1}\sigma_2\sigma_1^{-1}\sigma_2\sigma_1^{-2}\sigma_2^3$ $\sigma_1^{-3}\sigma_2^2$
100	$\sigma_1\sigma_2^{-2}\sigma_1^4\sigma_2^{-1}\sigma_1\sigma_2^{-4}\sigma_1\sigma_2^{-1}\sigma_1^3\sigma_2^{-1}\sigma_1^2\sigma_2^{-2}\sigma_1^4\sigma_2^{-1}\sigma_1\sigma_2^{-4}\sigma_1\sigma_2^{-1}\sigma_1^6\sigma_2^2\sigma_1\sigma_2\sigma_1\sigma_2^2\sigma_1^3\sigma_2^5\sigma_1^{-1}\sigma_2$ $\sigma_1^{-3}\sigma_2^3\sigma_2^5$ $\sigma_1^{-3}\sigma_2^3\sigma_2^5$
150	$\sigma_2\sigma_1^{-1}\sigma_2\sigma_1^{-1}\sigma_2^{-1}\sigma_1^2\sigma_2^{-1}\sigma_1\sigma_2^{-4}\sigma_1^2\sigma_2^{-2}\sigma_1^{-1}\sigma_2^{-1}\sigma_1^{-2}\sigma_2^{-4}\sigma_1^2\sigma_2^{-4}\sigma_1\sigma_2^{-1}\sigma_1^2\sigma_2^{-1}\sigma_1^{-1}\sigma_2\sigma_1^{-3}$ $\sigma_1^{-1}\sigma_1\sigma_2^{-1}\sigma_1^3\sigma_2^{-1}\sigma_1\sigma_2^{-4}\sigma_1^2\sigma_2^{-4}\sigma_1^3\sigma_2^{-3}$
200	$\sigma_2^{-2}\sigma_4\sigma_2^{-1}\sigma_1\sigma_2^{-4}\sigma_1\sigma_2^{-1}\sigma_1^3\sigma_2^{-1}\sigma_1^3\sigma_2^{-1}\sigma_1\sigma_2^{-4}\sigma_1\sigma_2^{-1}\sigma_1^4\sigma_2^{-1}\sigma_1\sigma_2^{-2}\sigma_1^2\sigma_2^3\sigma_1\sigma_2^{-1}\sigma_1\sigma_2^{-1}\sigma_1^{-5}$ $\sigma_2\sigma_2^{-1}\sigma_1^{-6}\sigma_2^{-2}\sigma_2^3$
250	$\sigma_1\sigma_2\sigma_1^{-2}\sigma_2\sigma_1^{-1}\sigma_2^{-1}\sigma_1\sigma_2^{-1}\sigma_1^{-3}\sigma_4\sigma_2^{-2}\sigma_4\sigma_1^{-1}\sigma_2\sigma_1^2\sigma_2\sigma_1^{-4}\sigma_2\sigma_2^{-2}\sigma_4\sigma_2^{-3}\sigma_1^{-1}\sigma_1\sigma_2^{-1}\sigma_1^{-1}\sigma_2^3$ $\sigma_1\sigma_2^{-1}\sigma_2^{-2}\sigma_1^{-1}\sigma_1\sigma_2^{-4}\sigma_1^2\sigma_2^{-4}\sigma_1\sigma_2^{-1}\sigma_1^4\sigma_2^{-1}\sigma_1^{-1}\sigma_2^{-1}\sigma_1^{-4}\sigma_2^{-1}\sigma_1^{-1}\sigma_2^{-1}\sigma_1^4\sigma_2^{-1}\sigma_1\sigma_2^{-4}\sigma_1^2\sigma_2^{-4}$ $\sigma_1\sigma_2^{-2}\sigma_1^{-1}\sigma_2^5\sigma_1^{-1}\sigma_2\sigma_1^{-4}\sigma_2^3\sigma_1^{-2}\sigma_2^{-4}\sigma_2^3\sigma_1^{-1}$

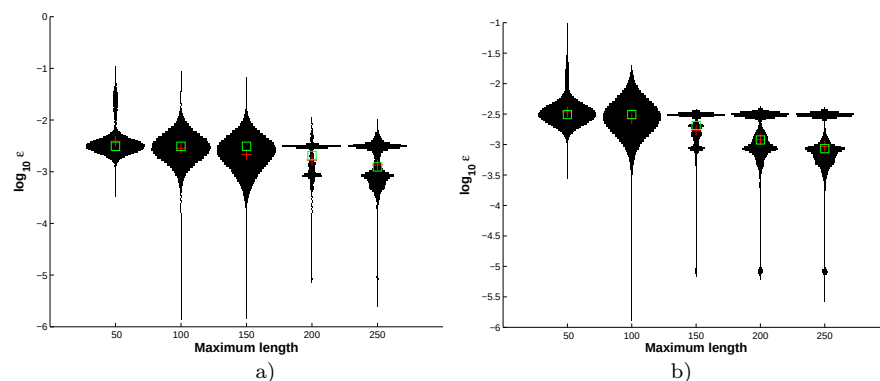
Table 1 Best braids found by the EDAs for each value of n .

Fig. 3 Results for target T_1 a) All EDAs variants with local optimizer (14400 runs), b) EDAs with local optimizer, recoding type II, and that use partial sampling type II (300 runs).

optimum has value $\log_{10}f(\epsilon) = -2.50785$ and the majority of the EDA runs can be trapped in this value. Figure 3b) reveals how a particular combination of the EDA's parameters can improve the results of the search. This is shown in detail in Table 2 that comprises all EDA variants that reached one of the best solutions in at least one of the 100 runs.

L	tf	$t\lambda$	ts	pm	50	100	150	200	250	tot	L	tf	$t\lambda$	ts	pm	50	100	150	200	250	tot
0	3	1	0	0	0	0	1	1	0	2	1	2	2	1	2	0	0	1	0	0	1
0	3	1	1	0	1	0	1	4	0	6	1	3	0	1	2	0	0	1	0	0	1
0	3	1	1	1	0	0	1	2	0	3	1	3	1	1	0	0	0	0	2	0	2
0	3	1	1	2	0	0	1	0	0	1	1	3	1	1	2	0	0	0	1	0	1
0	3	1	2	0	0	0	0	2	0	2	1	3	1	2	0	0	0	9	12	0	21
0	3	1	2	1	0	0	0	2	0	2	1	3	1	2	1	0	2	14	21	0	37
0	3	1	2	2	0	0	0	3	1	4	1	3	1	2	2	0	1	18	19	0	38
0	3	2	0	0	0	0	0	1	0	1	1	3	2	1	0	0	0	0	4	0	4
0	3	2	1	0	0	0	4	3	0	7	1	3	2	1	1	0	0	1	1	0	2
0	3	2	1	1	0	0	0	1	0	1	1	3	2	1	2	0	0	1	0	0	1
0	3	2	1	2	0	0	1	1	0	2	1	3	2	2	0	0	0	4	20	0	24
0	3	2	2	0	0	1	0	1	0	2	1	3	2	2	1	0	0	2	9	0	11
0	3	2	2	1	0	1	1	1	0	3	1	3	2	2	2	0	0	4	11	1	16
0	3	2	2	2	0	0	2	3	0	5	1	3	2	2	2	0	0	4	11	1	16

Table 2 Results target T_1 : EDA variants that reached one of the best solutions in at least one in the 100 experiments. L : Local optimizer, tf : Type of function and representation, $t\lambda$: type of λ value, ts : type of sampling, pm : probabilistic model.

There are a number of commonalities between the best EDA variants. Except in one case, all EDAs use recoding of type II. Similarly, except in one case, in all the variants $\lambda \in \{0.01, 0.05\}$. Except in two cases, the sampling method selected was partial sampling. The application of the local optimizer notably improved the results for $n \in \{150, 200\}$ but in terms of the best solution found it did not have an important influence for the other values of n .

As a summary, we recommend to use an EDA that adds the greedy search, and uses recoding of type II, partial sampling and the 1-order Markov model since it is less complex than the tree and results achieved using the two models are similar.

6.3.2 Improvement over other search methods

We compare our best EDA variant with the results achieved using a random search, the greedy local optimizer, and the GA introduced in [16]. For the random search, we randomly generated 10000 solutions and selected the best solution according to function f , $\lambda = 0.01$. The same experiment was repeated 100 times to select the 100 “best” solutions for $n \in \{50, 100, 150, 200, 250\}$.

A similar procedure was followed for the greedy local search. The local optimizer was applied to each of the 10000 solutions until no improvement was possible. For the GA, we used the results of the 100 GA runs analyzed in [16]. Since these results were obtained using solutions of different length, and with a different number of evaluations, care must be taken to interpret

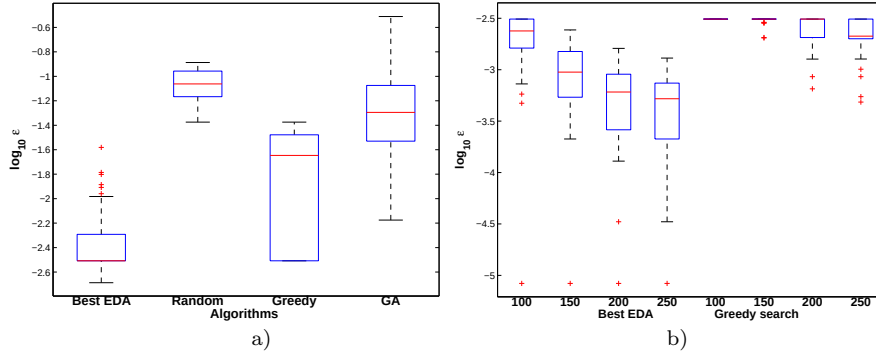


Fig. 4 a) Comparison between the Best EDA variant, the random search, the greedy local search and the GA for $n = 50$. b) Comparison between the Best EDA variant and the greedy local search $n \in \{100, 150, 200, 250\}$.

the differences. We only compare the GA results with the other algorithms for $n = 50$. Similarly, the results of the random search were very poor for $n > 50$ and we only include them in the comparison for $n = 50$. Results are shown in Figure 4a). In the boxplots, the central mark is the median, the edges of the box are the 25th and 75th percentiles, the whiskers extend to the most extreme data points not considered outliers, and outliers are plotted individually.

Using the 100 best solutions for each of the four algorithms, a multiple comparison test was applied to test the null hypothesis that samples corresponding to every pair of algorithms are drawn from the same population. The multiple comparison test uses the Tukey's honestly significant difference criterion. Every pair-wise comparison is based on the Kruskal-Wallis test, a nonparametric version of the classical one-way ANOVA. The significance criterion was $\alpha = 0.05$. The application of the test identified statistical differences between each pair of algorithms and they were ranked as: 1) Best-EDA, 2) Greedy, 3) GA, 4) Random.

The results of the comparison between the EDA and the greedy search for $n > 50$ are shown in Figure 4b). The application of the Kruskal-Wallis test ($\alpha = 0.05$) found significant differences between the EDA and the Greedy algorithm for all n . Furthermore, it can be seen in Figure 4b) that as n increases the algorithm is able to scale and find better solutions.

6.4 Results for the icosahedral group

By applying the algorithm to approximate the elements of the icosahedral group we intend to evaluate the performance of EDAs on a diverse set of gates which are relevant since they can be combined to obtain approximation of

other gates. In the experiments, we compare the results of the EDA with the best braid approximations obtained by brute-force² for $L = 44$.

We use the best parameters found for the EDA in the previous experiment, i.e. greedy search, recoding of type II, and partial sampling. We investigated the three probabilistic models in order to evaluate the influence of the type of interactions captured by the algorithm in the results. We set $\lambda = 0.05$ and $n = 120$. The other parameters of the EDAs were set as in the previous experiments.

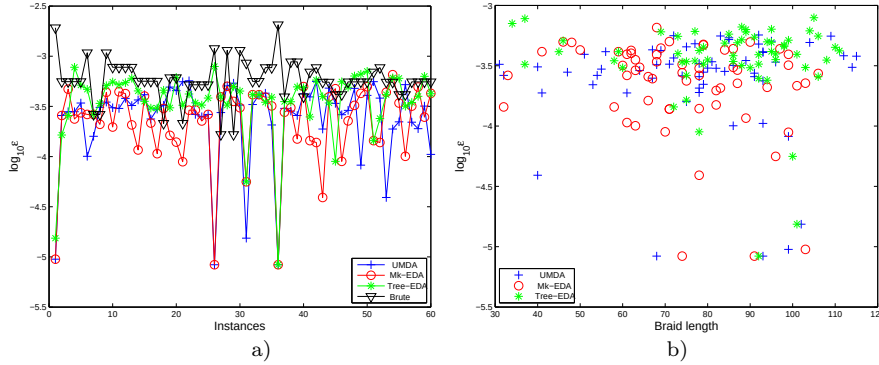


Fig. 5 Results icosahedral group: a) Error of the braid found the EDAs and those obtained by brute-force for $L = 44$. b) Comparison between EDAs with different probabilistic models in terms of the error and length of the best braid found.

Figure 5a) shows the errors of the approximations found by UMDA, Mk-EDA, and Tree-EDA for the 60 instances. It also shows the errors of the solutions found using the brute-force algorithm. UMDA was better than brute-force for 52 of the 60 instances, Mk-EDA was better than brute-force for 53 instances, and Tree-EDA was better for 42 instances. There were only 4 instances for which brute-force was better than the three EDAs. These results show, that even when the length of the braids considered for brute force is increased, EDAs can still beat the exhaustive search by searching in larger space $n > L$. While the ability of brute-search to improve the results by increasing L is limited, EDAs could further improve these results by increasing n , as shown for target T_1 .

Figure 5b) shows the errors and lengths of the best solutions found by EDAs for all instances. It can be seen in the figure that although the maximum possible length of the solutions is $n = 120$, several of the obtained braids have 70 or less components. There were differences in the length of the best solutions found by the three EDAs. The mean length of the best solutions for UMDA, Mk-EDA, and Tree-EDA were, respectively, 77.78, 74.22, and 84.06.

² Although in [4] results are presented for $L = 68$, available code (<https://sites.google.com/site/braidanyons/download>) only included results of brute-force up to $L = 44$.

The mean error obtained by the algorithms were, respectively, 0.31×10^{-3} , 0.28×10^{-3} , and 0.41×10^{-3} . For the icosahedral group, Mk-EDA obtains braids of higher accuracy and smaller length.

7 Conclusions

The problem of finding quantum gate approximations in topological quantum computing has been addressed using a variety of methods [3, 5, 4, 7, 13, 17, 35]. Many of these methods use, at least to a certain extent, brute-force search which is clearly unfeasible as the length of the braid is increased. The application of combinatorial optimization algorithms in this domain is promising. In particular, as shown in previous work [16, 30], evolutionary algorithms offer a better alternative to exhaustive methods. They can be tuned to adapt to the available computational resources (e.g. by conveniently setting the population size and number of generations) and information from exhaustive search in lower domains can be used to initialize the algorithms.

In this paper we have presented a detailed study of the use of EDAs for approximating a quantum gate as a product of braid generators. We have conducted extensive experiments on different target gates and compared with the results achieved with some of the most commonly used methods in this domain. We have shown that EDAs can find braids that provide accurate approximations. The best braids obtained with our EDAs for target T_1 have lengths up to 9 times shorter than those expected from braids of the same accuracy obtained with the Solovay-Kitaev algorithm and had not been previously reported to be found by the GA approach.

We have proposed the use of the icosahedral group as a benchmark for comparing quantum target approximation algorithms. Using this benchmark we have compared EDAs that use probabilistic models of different complexity. In general, our results show that the Mk-EDA can outperform the UMDA and Tree-EDA in terms of the accuracy and length of the obtained solutions. EDAs also outperform brute-force for $L = 44$.

As areas of future research, we plan to extend the analysis presented in this paper to extract more problem specific information that can be useful for designing more effective search methods. The application of a bi-objective optimization algorithm, able to simultaneously optimize the accuracy and length of the braids, is another area worth of research. By computing Pareto-set approximations for each element of the icosahedral group, multi-objective optimization would allow more flexible libraries of braids with much larger values L .

References

1. R. Armañanzas, I. Inza, R. Santana, Y. Saeys, J. L. Flores, J. A. Lozano, Y. Van de Peer, R. Blanco, V. Robles, C. Bielza, and P. Larrañaga. A review of estimation of distribution algorithms in bioinformatics. *BioData Mining*, 1(6):doi:10.1186/1756-0381-1-6, 2008.

2. S. Baluja and S. Davies. Using optimal dependency-trees for combinatorial optimization: Learning the structure of the search space. In D. H. Fisher, editor, *Proceedings of the 14th International Conference on Machine Learning*, pages 30–38, 1997.
3. N. E. Bonesteel, L. Hormozi, G. Zikos, and S. H. Simon. Braid topologies for quantum computation. *Physical review letters*, 95(14):140503, 2005.
4. M. Burrello, G. Mussardo, and X. Wan. Topological quantum gate construction by iterative pseudogroup hashing. *New Journal of Physics*, 13(2):025023, 2011.
5. M. Burrello, H. Xu, G. Mussardo, and X. Wan. Topological quantum hashing with the Icosahedral group. *Physical review letters*, 104(16):160502, 2010.
6. C. K. Chow and C. N. Liu. Approximating discrete probability distributions with dependence trees. *IEEE Transactions on Information Theory*, 14(3):462–467, 1968.
7. C. M. Dawson and M. A. Nielsen. The Solovay-Kitaev algorithm. *arXiv preprint quant-ph/0505030*, 2005.
8. J. S. De Bonet, C. L. Isbell, and P. Viola. MIMIC: Finding optima by estimating probability densities. In Mozer et al., editor, *Advances in Neural Information Processing Systems*, volume 9, pages 424–430. The MIT Press, Cambridge, 1997.
9. C. Echegoyen, A. Mendiburu, R. Santana, and J. A. Lozano. Toward understanding EDAs based on Bayesian networks through a quantitative analysis. *IEEE Transactions on Evolutionary Computation*, 16(2):173–189, 2012.
10. M. Henrion. Propagating uncertainty in Bayesian networks by probabilistic logic sampling. In J. F. Lemmer and L. N. Kanal, editors, *Proceedings of the Second Annual Conference on Uncertainty in Artificial Intelligence*, pages 149–164. Elsevier, 1988.
11. J. L. Hintze and R. D. Nelson. Violin plots: a box plot-density trace synergism. *The American Statistician*, 52(2):181–184, 1998.
12. A. Kaufman, G. Dror, I. Meilijson, and E. Ruppin. Gene expression of caenorhabditis elegans neurons carries information on their synaptic connectivity. *PLoS computational biology*, 2(12):e167, 2006.
13. A. Y. Kitaev. Fault-tolerant quantum computation by anyons. *Annals of Physics*, 303(1):2–30, 2003.
14. P. Larrañaga, H. Karshenas, C. Bielza, and R. Santana. A review on probabilistic graphical models in evolutionary computation. *Journal of Heuristics*, 18(5):795–819, 2012.
15. P. Larrañaga and J. A. Lozano, editors. *Estimation of Distribution Algorithms. A New Tool for Evolutionary Computation*. Kluwer Academic Publishers, 2002.
16. R. B. McDonald and H. G. Katzgraber. Genetic braid optimization: A heuristic approach to compute quasiparticle braids. *Physical Review B*, 87(5):054414, 2013.
17. R. Mosseri. A geometrical approach to $su(2)$ navigation with Fibonacci anyons. *Journal of Physics A: Mathematical and Theoretical*, 41(17):175302, 2008.
18. H. Mühlenbein and T. Mahnig. Evolutionary algorithms and the Boltzmann distribution. In K. A. DeJong, R. Poli, and J. Rowe, editors, *Foundation of Genetic Algorithms 7*, pages 133–150. Morgan Kaufmann, 2002.
19. H. Mühlenbein, T. Mahnig, and A. Ochoa. Schemata, distributions and graphical models in evolutionary optimization. *Journal of Heuristics*, 5(2):213–247, 1999.
20. H. Mühlenbein and G. Paaß. From recombination of genes to the estimation of distributions I. Binary parameters. In *Parallel Problem Solving from Nature - PPSN IV*, volume 1141 of *Lectures Notes in Computer Science*, pages 178–187, Berlin, 1996. Springer.
21. M. A. Nielsen and I. L. Chuang. *Quantum computation and quantum information*. Cambridge university press, 2010.
22. M. Pelikan and D. E. Goldberg. Hierarchical BOA solves Ising spin glasses and Max-Sat. IlliGAL Report No. 2003001, University of Illinois at Urbana-Champaign, January 2003.
23. M. Pelikan and D. E. Goldberg. Hierarchical BOA solves Ising spin glasses and Max-Sat. IlliGAL Report No. 2003001, University of Illinois at Urbana-Champaign, Illinois Genetic Algorithms Laboratory, Urbana, IL, January 2003.
24. M. Pelikan and H. G. Katzgraber. Analysis of evolutionary algorithms on the one-dimensional spin glass with power-law interactions. In *Proceedings of the 11th Annual conference on Genetic and evolutionary computation*, pages 843–850. ACM, 2009.

25. F. B. Pereira, P. Machado, E. Costa, A. Cardoso, A. Ochoa, R. Santana, and M. R. Soto. Too busy to learn. In *Proceedings of the 2000 Congress on Evolutionary Computation CEC-2000*, pages 720–727, La Jolla Marriott Hotel La Jolla, California, USA, July 2000. IEEE Press.
26. N. Read and E. Rezayi. Beyond paired quantum Hall states: parafermions and incompressible states in the first excited Landau level. *Physical Review B*, 59(12):8084, 1999.
27. R. Santana, C. Bielza, and P. Larrañaga. Conductance interaction identification by means of Boltzmann distribution and mutual information analysis in conductance-based neuron models. *BMC Neuroscience*, 13(Suppl 1):P100, 2012.
28. R. Santana, P. Larrañaga, and J. A. Lozano. Combining variable neighborhood search and estimation of distribution algorithms in the protein side chain placement problem. *Journal of Heuristics*, 14:519–547, 2008.
29. R. Santana, P. Larrañaga, and J. A. Lozano. Protein folding in simplified models with estimation of distribution algorithms. *IEEE Transactions on Evolutionary Computation*, 12(4):418–438, 2008.
30. R. Santana, R. B. McDonald, and H. G. Katzgraber. A probabilistic evolutionary optimization approach to compute quasiparticle braids. In *Proceedings of the 10th International Conference Simulated Evolution and Learning (SEAL-2014)*, pages 13–24. Springer, 2014.
31. S. D. Sarma, M. Freedman, and C. Nayak. Topological quantum computation. *Physics Today*, 59(7):32–38, 2006.
32. S. Shakya and R. Santana, editors. *Markov Networks in Evolutionary Computation*. Springer, 2012.
33. S. Shakya and R. Santana. A review of estimation of distribution algorithms and Markov networks. In S. Shakya and R. Santana, editors, *Markov Networks in Evolutionary Computation*, pages 21–37. Springer, 2012.
34. S. Shakya, R. Santana, and J. A. Lozano. A Markovianity based optimisation algorithm. *Genetic Programming and Evolvable Machines*, 13(2):159–195, 2012.
35. P. T. Trung, R. Van Meter, and C. Horsman. Optimising the Solovay-Kitaev algorithm. *arXiv preprint arXiv:1209.4139*, 2012.
36. S. Tsutsui, M. Pelikan, and D. E. Goldberg. Using edge histogram models to solve permutation problems with probabilistic model-building genetic algorithms. IlliGAL Report 2003022, University of Illinois at Urbana-Champaign, Illinois Genetic Algorithms Laboratory, Urbana, IL, 2003.
37. H. Xu and X. Wan. Constructing functional braids for low-leakage topological quantum computing. *Physical Review A*, 78(4):042325, 2008.